

Online Resource 1

PLENA implementation details and reproducible computational workflow

Article: Scalable Stochastic Characterization of Urban Drainage Networks: Applying PLENA to 239 Seoul Catchments

Journal	Stochastic Environmental Research and Risk Assessment
Authors	Changmin Park, Minsoo Seok, Yongwon Seo
Affiliation	Department of Civil Engineering, Yeungnam University, Gyeongsan, Republic of Korea
Corresponding author	Yongwon Seo (yongwon.seo@yu.ac.kr)
Repository	https://github.com/seokminsu99-byte/FMT_plug_PLENA

Resource caption

PLENA implementation details and reproducible computational workflow. This PDF contains supplementary figures, descriptions of the input-file structure and flow-direction coding, compilation and execution procedures, candidate generation and acceptance rules, output-file descriptions, implementation notes, and additional computational-performance results.

1. Scope

PLENA (Program for Large-scale Evaluation of Network Analysis) is the C++17 implementation used in the accompanying study for stochastic Gibbs-model drainage-network generation, width-function calculation, Nash-Sutcliffe efficiency (NSE) comparison, and representative beta-class estimation. This resource documents the checked source implementation and the workflow needed to interpret the supplementary figures and output files.

The associated source code is available at https://github.com/seokminsu99-byte/FMT_plug_PLENA. The repository contains the checked C++ source, example input, direction-code explanation, license, citation metadata, and the two SERRA online-resource files.

2. Reproducible computational workflow

2.1 Build requirements

A compiler supporting C++17 and standard thread support is required. The repository build command is:

```
g++ -O2 -std=c++17 -pthread src/main_EN_ver.cpp -o PLENA
```

The transition-kernel diagnostic can be run with `./PLENA --self-test`.

2.2 Input-file structure

The plain-text input contains: (1) the number of rows and columns, (2) the one-based outlet row and column, and (3) the drainage-direction matrix. Direction codes are 0 = inactive/no pipe, 1 = east, 2 = south, 3 = west, and 4 = north. The outlet must lie within the active-cell mask, and every active cell must reach it.

2.3 Execution

Run PLENA with an input path (for example, `PLENA example.txt`) or start it without an argument and select a listed text file. The user enters the exponent k in $\beta = 10^k$ and the update coefficient α in $I = \alpha \text{ nm}$; entering $\alpha = 0$ selects the study setting $\alpha = 10$. The default seed is generated automatically, while an optional fixed seed may be supplied as the second command-line argument for an exactly repeatable diagnostic run.

Width-function analysis is optional. When batch width functions are requested, the user selects the candidate exponent range and replicate count; a thread cap of 0 requests automatic selection from `hardware_concurrency()`.

2.4 Output files

The primary result text file contains sampler metadata, the generated direction matrix, travel-time matrix, flow-accumulation matrix, and $q(t)$. Optional width-function analysis writes FD and LS matrices, a width-functions CSV file, and an NSE text report. The NSE report provides candidate-class summaries used to compare simulated mean width functions with the observed width function.

3. Checked implementation details

3.1 Contiguous matrix storage

The checked Matrix type stores cells in a single row-major integer vector. This layout improves spatial locality during repeated neighbor access in candidate updates, reachability checks, travel-time calculations, and width-function construction. Figure S1 illustrates the cache-locality rationale.

3.2 Candidate generation, screening, and outlet reachability

At each stochastic update, PLENA uniformly selects one non-outlet active cell and one of the three flow directions other than its current direction, thereby generating a candidate network D_{new} . Boundary violations, movement to an inactive cell, and immediate two-cycle conflicts are screened before a breadth-first outlet search checks whether every active cell in D_{new} remains outlet-reachable. A failed proposal leaves the current network D unchanged as a self-transition, and the proposal attempt is counted toward the update budget.

3.3 Transition and acceptance rule

For an outlet-reachable candidate, PLENA computes $\Delta H = H(D_{\text{new}}) - H(D)$. The general off-diagonal transition rule is $p = (1/r) \min\{1, \exp(-\beta \Delta H)\}$, where $r = 3N_u$ and N_u is the number of non-outlet active cells. The uniform cell-direction selection supplies $1/r$; conditional acceptance of the selected candidate therefore uses $a = \min\{1, \exp(-\beta \Delta H)\}$. A uniform random number u is drawn on $[0,1)$, and D is replaced by D_{new} when $u < a$.

3.4 Scale-proportional stochastic updates

The implementation uses the size-normalized update budget $I = \alpha nm$, where n and m are the grid dimensions. The checked study configuration uses $I = 10nm$. Every proposal attempt, including locally screened and outlet-unreachable proposals, contributes to I .

3.5 Parallel batch execution

Candidate beta classes and stochastic realizations are independent tasks. PLENA uses a C++ worker pool with atomic task indexing and an optional user thread cap. Updates within an individual stochastic chain remain sequential. Figure S2 documents automatic 24-thread execution and high processor utilization in the captured run.

3.6 Computational-performance comparison

Figure S3 compares generation times for the conventional MATLAB implementation and PLENA over the benchmark network sizes reported in the study. The logarithmic time axis shows the increasing runtime separation as network size grows. The reported acceleration applies to the tested hardware and configurations and is not a hardware-independent guarantee.

3.7 Interpretation boundary

PLENA and the beta class describe drainage-network structure through outlet-referenced flow-distance patterns. They do not by themselves represent rainfall forcing, pipe capacity, pumps, downstream water levels, surface storage, inundation depth, or flood probability.

Figure S1. Contiguous storage and cache locality

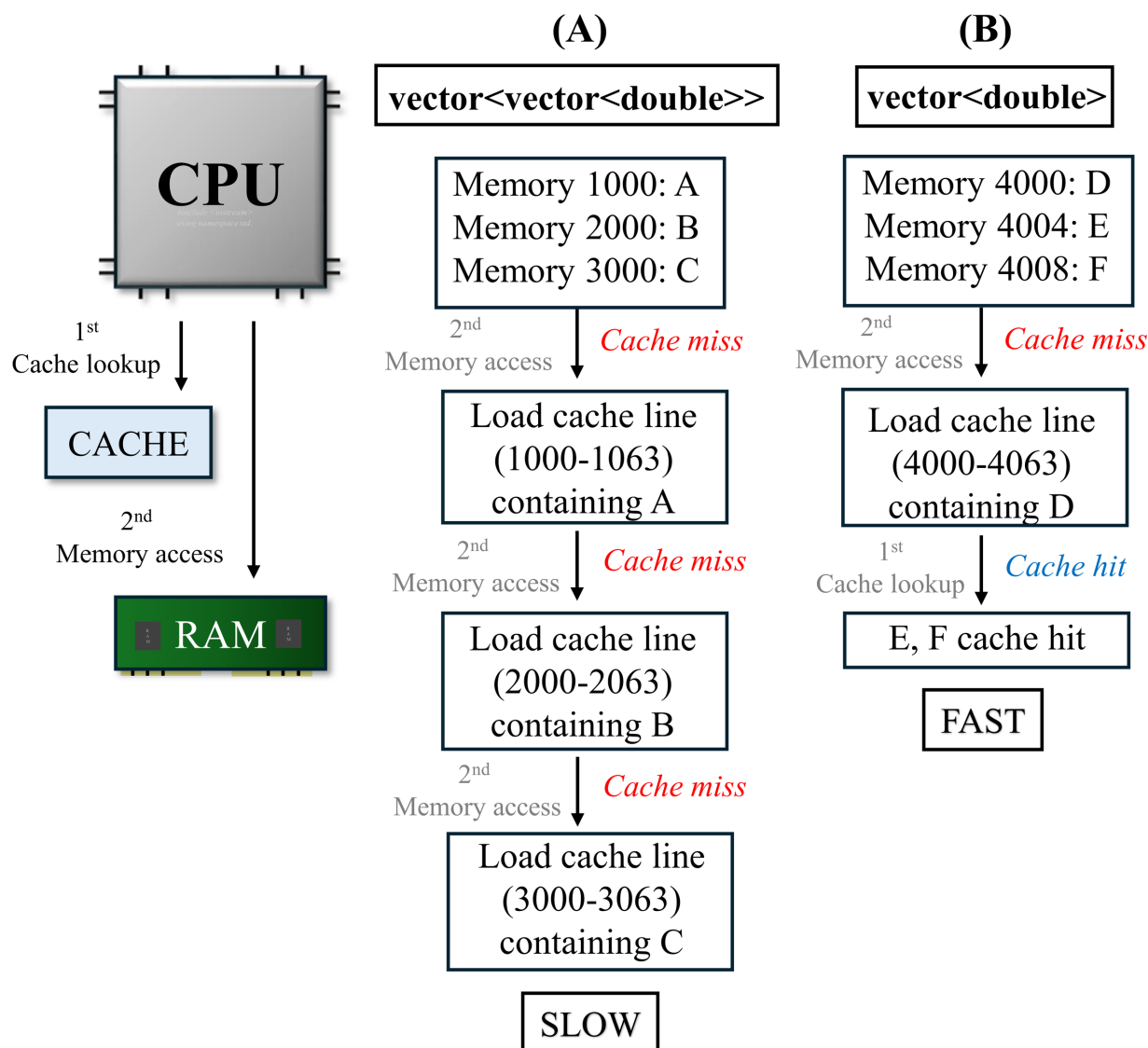


Figure S1. Schematic comparison of nested and contiguous memory layouts. Separate nested containers can require repeated cache-line loads during neighboring-cell access, whereas row-major contiguous storage places adjacent cell values together and improves cache locality. This figure explains the Matrix storage strategy used in PLENA.

Figure S2. Parallel execution

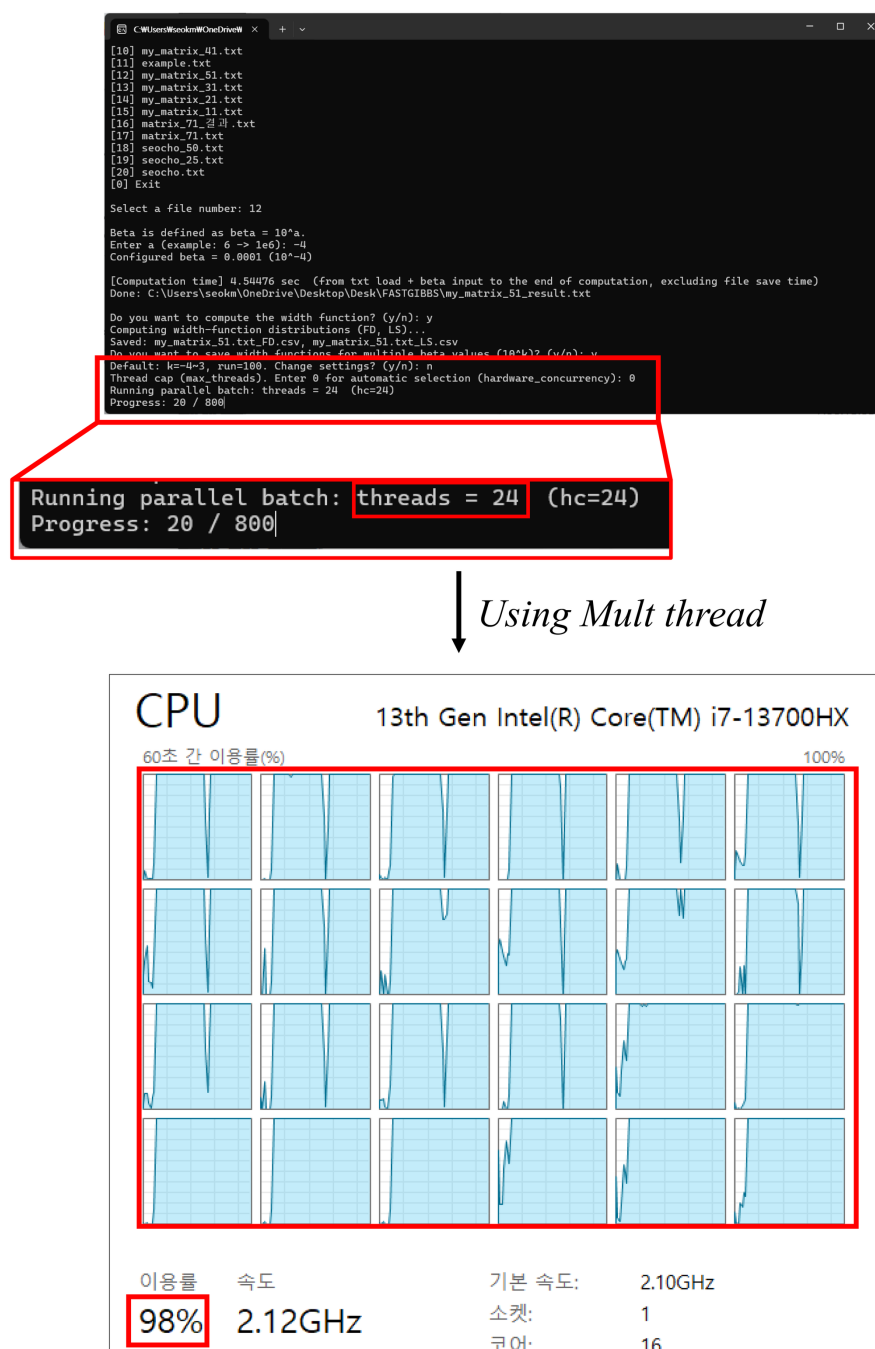


Figure S2. PLENA console output and processor utilization during parallel width-function estimation. The captured run reports automatic use of 24 worker threads, while the operating-system performance panel shows high processor utilization. The screenshot documents parallel execution across independent beta-class and replicate tasks; utilization may vary with hardware and workload.

Figure S3. Computational-performance comparison

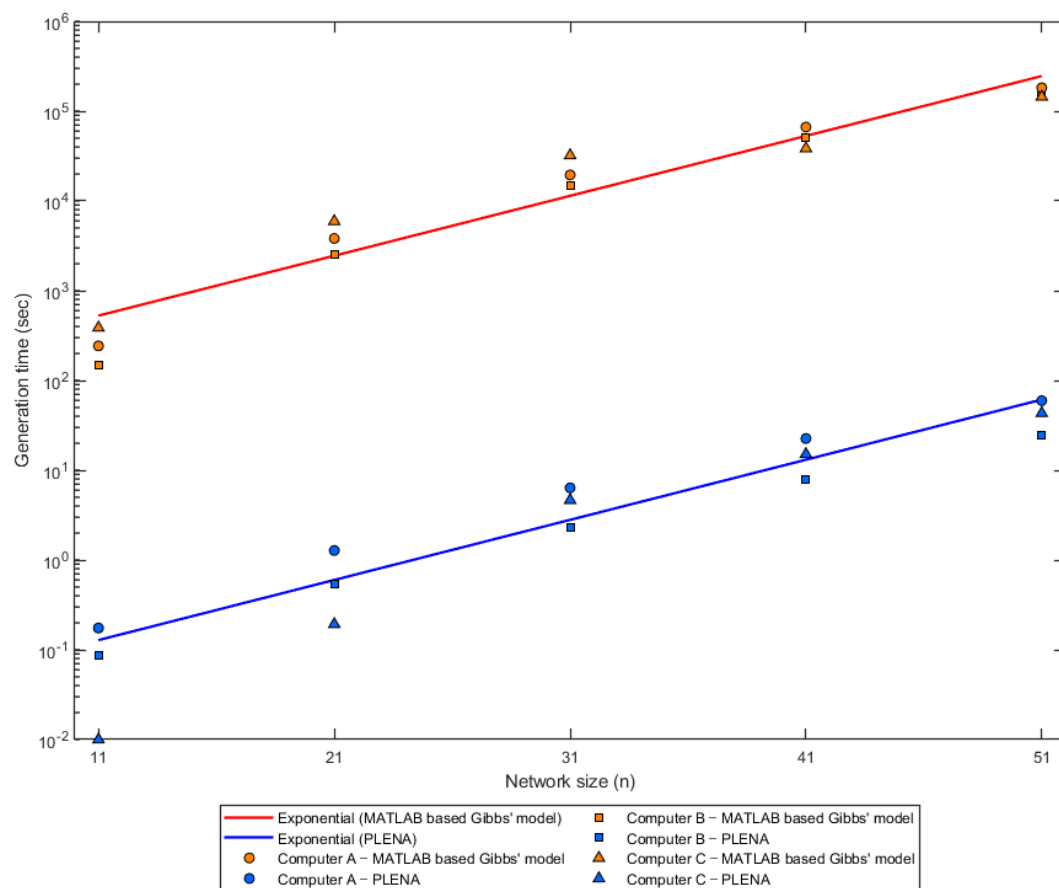


Figure S3. Generation-time comparison between the MATLAB-based Gibbs-model implementation and PLENA across the tested network sizes. The y-axis is logarithmic. PLENA remains below the MATLAB implementation across the benchmark sizes, and the separation grows with network size. Values represent the tested configurations and computing environment reported in the study.